

## Flashing Nvidia Jetson Devices

### Using this guide:

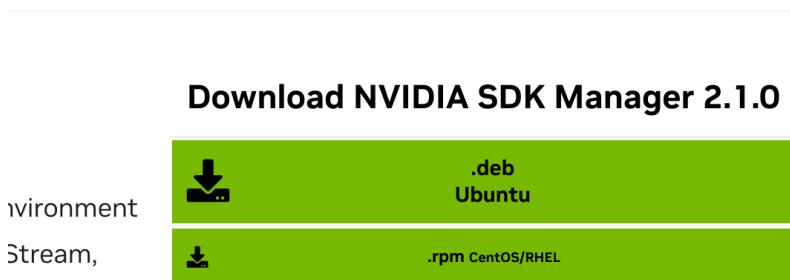
Lines written in **green** indicate text to be input into the Linux terminal. Lines preceded by “#” are comments that are for labeling the purposes of certain steps.

### 1. Requirements:

- A separate computer with Linux already installed.

### 2. Download SDK Manager:

- On a separate computer visit: <https://developer.nvidia.com/sdk-manager>



- Click on Join in the upper right corner and complete the process for signing up.
- Once you have signed up, click on the “.deb Ubuntu” button to download the SDK Manager.

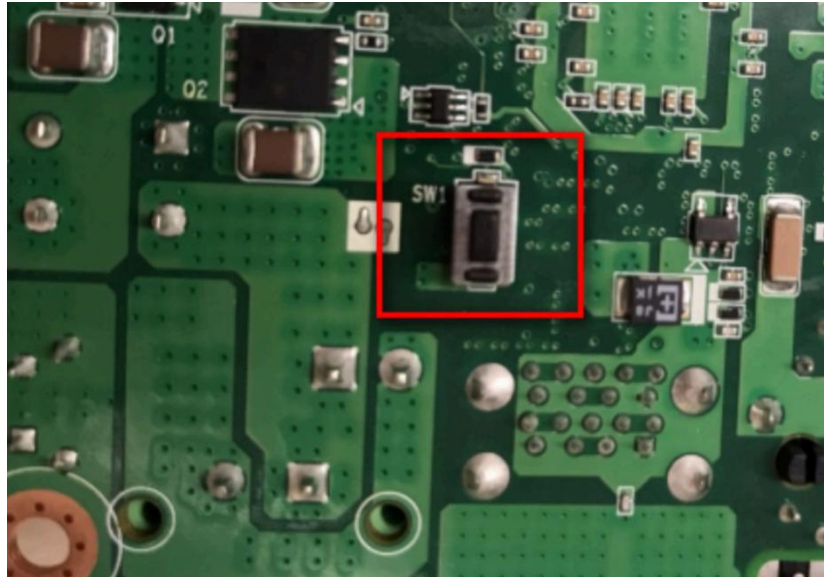
### 3. Installation:

- `cd ~/Downloads/`
- # Install the SDK Manager**
- `sudo dpkg -i sdkmanager_2.1.0-11682_amd64.deb`
- # Open SDK Manager**
- `sdkmanager`

### 4. Put Jetson into Recovery Mode:

- If Jetson can be turned on: `sudo reboot --force forced=recovery`
- If Jetson cannot be turned on:

- Manually put it into recovery mode by holding down the SW1 button on the back of the motherboard while powering on.



#### 5. Connect Jetson to Host Computer:

- Use a micro-USB cable to connect Jetson to the host computer.

#### 6. Detection and Selection:

- SDK Manager should automatically detect the Jetson device in recovery mode.
- If not detected, navigate to "Target Hardware" and select your device.

#### 7. Flash Jetson:

- Select the newest version of Jetpack and proceed to Step 2.
- Choose all necessary downloads, accept terms, and continue to Step 3.
- Set a new username and password, ensure correct storage device type, and initiate flashing.

#### 8. Completion:

- Once the download and flashing process is complete, the Jetson should boot up.

### SOP for Installing YOLO v3 on NVIDIA Jetson Devices

The following steps go over how to install YOLO v3 on NVIDIA Jetson devices, including the necessary dependencies such as jtop, cuDNN, OpenCV, and CUDA.

## 1. Install jtop

(1). Make sure the jetson device is connected to the internet.

(2). Update the system and install dependencies and recommended packages:

☐ `sudo apt update`

☐ `sudo apt upgrade`

☐ `sudo reboot`

# Python dependencies

☐ `sudo apt-get install python3-dev python3-numpy python3-pip`

☐ `sudo apt install python3-testresources`

(3). Install jetson-stats package using pip:

☐ `sudo pip3 install -U jetson-stats`

(4). After rebooting, run jtop:

☐ `sudo jtop`

(5). Nvidia-jet should install most of the required dependencies except Opencv.

## 2. Result of terminal display configuration

```
jtop 4.2.8 - (c) 2024, Raffaello Bonghi [raffaello@rnext.it]
Website: https://rnext.it/jetson_stats

Platform
XXX]chine: aarch64
System: Linux
Distribution: Ubuntu 22.04 Jammy Jellyfish
Release: 5.15.136-tegra
Python: 3.10.12

Serial Number: [s]XX CLICK TO READ
Hardware
Model: NVIDIA Jetson Orin NX Engin
699-level Part Number: 699-13767-0
P-Number: p3767-0004
Module: NVIDIA Jetson Orin Nano (4
SoC: tegra234
CUDA Arch BIN: 8.7
L4T: 36.3.0
Jetpack: 6.0

Libraries
CUDA: 12.2.140
cuDNN: 8.9.4.25
TensorRT: 8.6.2.3
VPI: 3.1.5
Vulkan: 1.3.204
OpenCV: 4.10.0 with CUDA: YES

Hostname: ubuntu
Interfaces
eth0: 10.1.2.5

1ALL 2GPU 3CPU 4MEM 5ENG 6CTRL 7INFO Quit (c) 2024, RB
```

\* Install opencv: (go to this link if you have any trouble)

<https://gist.github.com/minhhieutruong0705/8f0ec70c400420e0007c15c98510f133>

# Required Packages

☐ `sudo apt-get install libjpeg-dev libpng-dev libtiff-dev`

☐ `sudo apt-get install build-essential cmake pkg-config unzip`

yasm git checkinstall # generic tools

# Image I/O

○ sudo apt-get install cmake git libgtk2.0-dev pkg-config  
libavcodec-dev libavformat-dev libswscale-dev # required

#Video and audio

○ sudo apt install libavcodec-dev libavformat-dev

libswscale-dev libavresample-dev

○ sudo apt install libgstreamer1.0-dev

libgstreamer-plugins-base1.0-dev

○ sudo apt install libxvidcore-dev x264 libx264-dev

libfaac-dev libmp3lame-dev libtheora-dev

○ sudo apt install libfaac-dev libmp3lame-dev libvorbis-dev

○ sudo apt-get install nvidia-jetpack

# For speed codec

○ sudo apt install libopencore-amrnb-dev libopencore-amrwb-dev

# Camera programming interface

○ sudo apt-get install libdc1394-22 libdc1394-22-dev

libxine2-dev libv4l-dev v4l-utils

○ cd /usr/include/linux

○ sudo ln -s -f ../libv4l1-videodev.h videodev.h

○ sudo ldconfig

○ cd ~

# GTK for OpenCV highgui

○ sudo apt-get install libgtk-3-dev

# Parallelism C++ for CPU

○ sudo apt-get install libtbb2 libtbb-dev

# OpenCV optimization

○ sudo apt-get install libatlas-base-dev gfortran

# Remove opencv-python-headless

○ `python3 -m pip uninstall opencv-python-headless`

# Remove python3-opencv

○ `sudo apt-get remove python3-opencv`

### 3. Install OpenCV

(1). Copy python libraries to c++ compiler

○ `sudo cp -r /usr/include/python3.10/ /usr/include/`

(2). Download github repositories

○ `cd ~`

○ `git clone https://github.com/opencv/opencv.git`

○ `git clone https://github.com/opencv/opencv_contrib.git`

(3). Create build folder

○ `cd ~`

○ `mkdir opencv_build`

○ `cd opencv_build`

(4). Add flags

```
cmake -D CMAKE_BUILD_TYPE=Release \  
-D CMAKE_INSTALL_PREFIX=/usr/local \  
-D OPENCV_EXTRA_MODULES_PATH=~/.opencv_contrib/modules/ \  
-D PYTHON3_EXECUTABLE=/usr/bin/python3 \  
-D PYTHON3_INCLUDE_DIR=/usr/include/python3.8/ \  
-D PYTHON3_INCLUDE_DIR2=/usr/include/x86_64-linux-gnu/python3.8/ \  
-D PYTHON3_LIBRARY=/usr/lib/x86_64-linux-gnu/libpython3.8.so \  
-D PYTHON3_NUMPY_INCLUDE_DIRS=/usr/lib/python3/dist-  
packages/numpy/core/include/ \  
-D OPENCV_GENERATE_PKGCONFIG=ON \  
-D OPENCV_PC_FILE_NAME=opencv.pc \  
-D WITH_CUDA=ON \  
-D WITH_CUDNN=ON \  
-D OPENCV_DNN_CUDA=ON
```

```
-D CUDA_ARCH_BIN=8.6 \  
-D ENABLE_FAST_MATH=ON \  
-D CUDA_FAST_MATH=ON \  
-D WITH_CUFFT=ON \  
-D WITH_CUBLAS=ON \  
-D WITH_V4L=ON \  
-D WITH_OPENCL=ON \  
-D WITH_OPENGL=ON \  
-D WITH_GSTREAMER=ON \  
-D WITH_TBB=ON ../opencv
```

(5). Build opencv

- `make -j4`(4 is the number of core in the processor used to compile I recommend using less then the max)
- # Run “make -j4” until the operation takes less than a minute to finish.

(6). Install opencv

- `sudo make install`
- # Run “sudo make install” until the operation finishes in less than 1 minute.
- `sudo /bin/bash -c 'echo "/usr/local/lib" >> /etc/ld.so.conf.d/opencv.conf'`
- `sudo ldconfig`

(7). Verify opencv

- `jtop`
- Press “7”
- Verify the OpenCV row has a green “YES”

```
jtop 4.2.8 - (c) 2024, Raffaello Bonghi [raffaello@rnext.it]
Website: https://rnext.it/jetson_stats

Platform                               Serial Number: [s]XX CLICK TO READ
XXX]chine: aarch64                    Hardware
System: Linux                         Model: NVIDIA Jetson Orin NX Engin
Distribution: Ubuntu 22.04 Jammy Jellyfish 699-level Part Number: 699-13767-0
Release: 5.15.136-tegra                P-Number: p3767-0004
Python: 3.10.12                       Module: NVIDIA Jetson Orin Nano (4
                                         SoC: tegra234
Libraries                               CUDA Arch BIN: 8.7
CUDA: 12.2.140                        L4T: 36.3.0
cuDNN: 8.9.4.25                       Jetpack: 6.0
TensorRT: 8.6.2.3
VPI: 3.1.5                            Hostname: ubuntu
Vulkan: 1.3.204                       Interfaces
OpenCV: 4.10.0 with CUDA: YES          eth0: 10.1.2.5

1ALL 2GPU 3CPU 4MEM 5ENG 6CTRL 7INFO Quit (c) 2024, RB
```

## 4. Installing YOLO v3

### What is YOLO?

YOLO (You Only Look Once) is an object detection algorithm that uses a Deep Neural Network (DNN) framework to run. The DNN framework used for YOLO is called Darknet.

### Installation Steps:

#### (1). Update the libraries:

- `sudo apt-get update`

#### (2). Export CUDA path: (make sure to specify the version of cuda if you have another version)

- `export PATH=/usr/local/cuda-12.2/bin${PATH:+:${PATH}}`
- `export LD_LIBRARY_PATH=/usr/local/cuda-12.2/lib64${LD_LIBRARY_PATH:+:${LD_LIBRARY_PATH}}`

#### (3). Download Darknet and YOLO:

- `cd ~`
- `git clone https://github.com/AlexeyAB/darknet`
- `cd darknet`
- `wget https://pjreddie.com/media/files/yolov3.weights`
- `wget https://pjreddie.com/media/files/yolov3-tiny.weights`

#### (4). Enable the GPU, CUDNN, and OPENCV:

## Edit the Makefile to enable GPU, CUDA, and OpenCV:

- sudo vi Makefile

## # Quick guide to using VIM.

To edit text first press I to enter INSERT Mode

```
#include <stdio.h>

unsigned int compare(unsigned int x, unsigned int y)
{
    // Masks the most significant byte of x and the
    // three least significant bytes of y and
    // combines them.
    return (x & 0xFF000000) | (y & 0x00FFFFFF);
}

-- INSERT --
```

Insert appears at bottom. If you are having trouble getting to insert mode hit escape then press “i” again. To quit and save, press escape to enter neutral mode (-insert- is no longer at the bottom) then hold shift and press z twice (shift + ZZ).

■ Set the values:

■ GPU=1

■ CUDNN=1

■ OPENCV=1

■ Leave the rest as it is.

### (5). Compile Darknet:

○ make

- (if you get any errors retry exporting the paths and try again)

## How to Run YOLO v3?

You can run YOLO from the Linux terminal. Access the Darknet folder and choose one of the following commands depending on the type of detection you want to perform:

**(1). Image detection:**

- Edit “dog.jpg” with the path of your image:

```
./darknet detector test cfg/coco.data cfg/yolov3.cfg yolov3.weights -ext_output
```



dog.jpg

## (2). Detection from Webcam:

- The 0 at the end of the line is the index of the Webcam:

■ `./darknet detector demo cfg/coco.data cfg/yolov3.cfg yolov3.weights -c 0`

## (3). Detection from a Video file:

- Edit “test.mp4” with the path of your video file:

■ `./darknet detector demo cfg/coco.data cfg/yolov3.cfg yolov3.weights -ext_output test.mp4`

By following these steps, you will have all necessary dependencies installed and be able to run YOLO v3 on your NVIDIA Jetson device. For more detailed instructions and additional options, refer to the official documentation provided by each software vendor.

## Training YOLO v3

### Find and Annotate a Dataset

If you do not already have one you can find a quick one here.

1. Go to <https://public.roboflow.com/> to find a free dataset.
2. Download the dataset in "YOLO Darknet" format as a zip file.

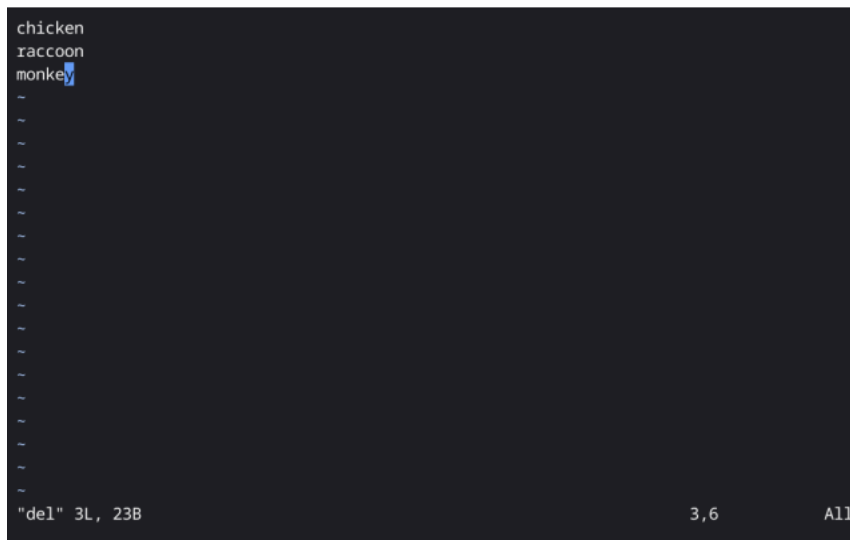
### Training the AI (Example: Laccoon Recognition)

#### (1). Create custom\_data Directory

- `cd ~/`
- `mkdir training`
- `cd training`
- `mkdir raccoon` <Or an appropriate name for the date e.g. raccoon>
- `cd raccoon`
- Unzip the dataset and move its contents into the raccoon directory.

#### (2). Create custom.names File

- `cd ~/training/ raccoon`
- `vim custom.names`
- List the labels of objects, each on a new line. See picture for reference.



After adding the label name, save it by first pressing the "ESC key", then simultaneously pressing "Shift" and ";", followed by typing "wq" and pressing Enter. (Alternatively, you can directly add the label names in the "custom.names" file.)

### (3). Create train.txt and test.txt Files

- `cd ~/training/raccoon/`
- `find ./train/ -name *.jpg -exec realpath {} \; > train.txt`
- `find ./valid/ -name *.jpg -exec realpath {} \; > valid.txt`

This is an example of the paths for train.txt and test.txt.

/home/ubuntu/training/raccoon/train/raccoon-108\_jpg.rf.b1c0928dfe114cfbd882e12141203f21.jpg

### (4). Create Backup Directory

- `cd ~/training/raccoon/`
- `mkdir backup`

# Create detector.data file

- `vim detector.data`

○ Include information about the number of classes, paths for train.txt, test.txt, custom.names, and the backup directory.

This is an example of the detector.data file (make sure to include the train =, valid =, ect.):

classes= 1

train = /home/ubuntu/training/raccoon/train.txt

valid = /home/ ubuntu /training/ raccoon /valid.txt

names = /home/ ubuntu /training/ raccoon /custom.names

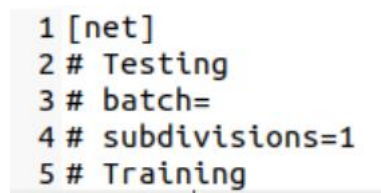
backup = /home/ ubuntu /training/ raccoon /backup/

#### (5). Prepare YOLOv3 Configuration File

- Create a `cfg` directory inside the `raccoon` directory.
- Copy the `yolov3.cfg` file from the darknet `cfg` directory.
- paste it to the `cfg` directory and Rename it to `yolov3custom.cfg` .

#### (6). Modify the Configuration File

○ Open the `yolov3custom.cfg` file located in the `cfg` folder, add the test parameters, and uncomment the training parameters as shown in the image below. .



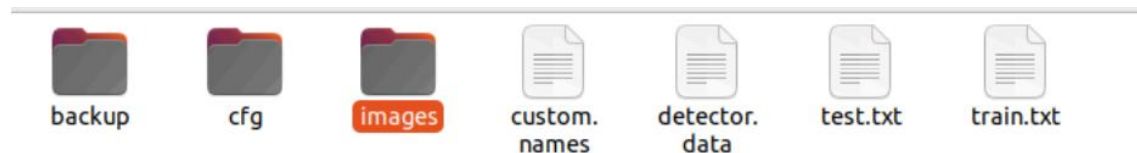
```
1 [net]
2 # Testing
3 # batch=
4 # subdivisions=1
5 # Training
```

- a. Add "#" at the beginning of lines 3 and 4, and remove "#" from the beginning of lines 6 and 7.
- b. Set the batch size (line 6) to 2. The batch parameter is used to update the weights by iterating over a small set of images. The initial value is 64.
- c. Set the sub-batch size (line 7) to 8.
- d. Adjust the `max_batches` (line 20). The `max_batches` parameter determines the maximum number of iterations the network should train. It is calculated as `max_batches = classes * 2000`. For a single class, the value would be 2000.
- e. Update the steps (line 22) to 80% and 90% of `max_batches`. For example, if `max_batches = 2000`, then the steps would be 1600 and 1800.
- f. Update the `classes` parameter in the `[yolo]` layers (lines 610, 696, and 783) to match the number of classes you are using. For example, for raccoon detection with one class, set it to 1.
- g. Update the `filters` parameter in the `[yolo]` layers (lines 603, 689, and 776) based on

the number of classes:  $\text{filters} = (\text{classes} + 5) * 3$ . For a single class, set  $\text{filters} = 18$ .

h. Now you can save and exit the cfg folder.

This is an example of the completed custom\_data folder



### (7). Download Pre-trained Weights

○ Run this command in the terminal:

○ `wget https://pjreddie.com/media/files/darknet53.conv.74`

### (8). Begin Training

○ Run this command in the terminal from the darknet directory to begin training:

○ `./darknet detector train <path to custom detector.datafile> <path to yolo-custom.cfg file> darknet53.conv.74 -dont_show`

例如: `./darknet detector train ~/training/raccoon/detector.data`

`~/training/raccoon/cfg/yolov3custom.cfg darknet53.conv.74 -dont_show`

○ If training stops at any time, it can be continued from where it left off instead of starting from scratch using this command.

○ `./darknet detector train <path to custom detector.data file> <path to yolo-custom.cfg file> <path to .weights file in backup folder> -dont_show`

### (9). Test mean average precision of specific weights

○ `./darknet detector map <path to custom detector.data file> <path to yolo-custom.cfg file> <path to .weights file in backup folder> -dont_show`

例如:

`./darknet detector map ~/training/raccoon/detector.data`

`~/training/raccoon/cfg/yolov3custom.cfg`

`~/training/raccoon/backup/yolov3custom_last.weights -dont_show`

### (10). Run your model

○ Running the model on an image:

○ `./darknet detector test <path to custom detector.data file> <path to yolo-custom.cfg`

file> <path to final.weights file in backup folder> -ext\_output <path to appropriate image for testing>

例如: ./darknet detector test ~/training/raccoon/detector.data

~/training/raccoon/cfg/yolov3custom.cfg

~/training/raccoon/backup/yolov3custom\_last.weights - ext\_output

~/training/raccoon/test/xxx.jpg

- Running the model on a video:

- ./darknet detector demo <path to custom detector.data file> <path to yolo-custom.cfg file> <path to final.weights file in backup folder> -ext\_output <path to appropriate video for testing>

- Running the model on a webcam:

- ./darknet detector demo <path to custom detector.data file> <path to yolo-custom.cfg file> <path to final.weights file in backup folder> -c 0

## Training the AI (Example: Pill multi-class recognition)

### (1). Create custom\_data Directory

- cd ~/

- mkdir training

- cd training

- mkdir raccoon <Or an appropriate name for the date e.g. raccoon>

- cd pill

- Unzip the dataset and move its contents into the raccoon directory.

### (2). Create custom.names File

- cd ~/training/pill

- vim custom.names

- In the pill dataset, there is a labels file. Copy the names of all 8 types into the custom.names file. The following are the 8 types:

- ☐ Cipro 500

- ☐ Ibuphil 600 mg

- ☐ Ibuphil Cold 400-60

- ☐ Xyzall 5mg
- ☐ blue
- ☐ pink
- ☐ red
- ☐ white

After adding the label names, save the file by first pressing the "ESC key", then simultaneously pressing "Shift" and ";", followed by typing "wq" and pressing Enter. (Alternatively, you can directly add the label names in the custom.names file.)

### (3). Create train.txt and test.txt Files

- `cd ~/training/pill/`
- `find ./train/ -name *.jpg -exec realpath {} \; > train.txt`
- `find ./valid/ -name *.jpg -exec realpath {} \; > valid.txt`

This is an example of the paths for train.txt and test.txt.

/home/ubuntu/training/pill/train/pill-108.jpg.rf.b1c0928dfe114cfbd882e12141203f21.jpg

### (4). Create Backup Directory

- `cd ~/training/pill/`
- `mkdir backup`

# Create detector.data file

- `vim detector.data`

○ Include information about the number of classes, paths for train.txt, test.txt, custom.names, and the backup directory.

This is an example of the detector.data file (make sure to include the train =, valid =, ect.):

classes= 8

train = /home/ubuntu/training/pill/train.txt

valid = /home/ ubuntu /training/pill/valid.txt

names = /home/ ubuntu /training/pill/custom.names

backup = /home/ ubuntu /training/pill/backup/

### (5). Prepare YOLOv3 Configuration File

- Create a `cfg` directory inside the `pill` directory.

- Copy the `yolov3.cfg` file from the darknet `cfg` directory.
- paste it to the `cfg` directory and Rename it to `yolov3custom.cfg`.

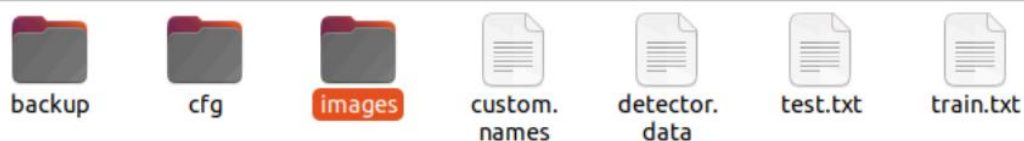
#### (6). Modify the Configuration File

- Open the `yolov3custom.cfg` file located in the `cfg` folder, add the test parameters, and uncomment the training parameters as shown in the image below.

```
1 [net]
2 # Testing
3 # batch=
4 # subdivisions=1
5 # Training
```

- Add "#" at the beginning of lines 3 and 4, and remove "#" from the beginning of lines 6 and 7.
- Set the batch size (line 6) to 2. The batch parameter is used to update weights by iterating over a small set of images. The initial value is 64.
- Set the sub-batch size (line 7) to 8.
- Adjust the `max_batches` (line 20). The `max_batches` parameter determines the maximum number of iterations the network should train. It is calculated as `max_batches = classes * 2000`. For a single class, the value would be 2000. In this case, set it to 16000.
- Update the steps (line 22) to 80% and 90% of `max_batches`. For example, if `max_batches = 16000`, then the steps would be 12800 and 14400.
- Update the `classes` parameter in the `[yolo]` layers (lines 610, 696, and 783) to match the number of classes you are using. For example, for pill recognition with 8 classes, set it to 8.
- Update the `filters` parameter in the `[yolo]` layers (lines 603, 689, and 776) based on the number of classes: `filters = (classes + 5) * 3`. For 8 classes, set `filters = 39`.
- Now you can save and exit the `cfg` folder.

This is an example of the completed `custom_data` folder



### (7). Download Pre-trained Weights

- Run this command in the terminal:

- `wget https://pjreddie.com/media/files/darknet53.conv.74`

### (8). Begin Training

- Run this command in the terminal from the darknet directory to begin training:

- `./darknet detector train <path to custom detector.datafile> <path to yolo-custom.cfg file> darknet53.conv.74 -dont_show`

例如: `./darknet detector train ~/training/pill/detector.data`

`~/training/pill/cfg/yolov3custom.cfg darknet53.conv.74 -dont_show`

- If training stops at any time, it can be continued from where it left off instead of starting from scratch using this command.

- `./darknet detector train <path to custom detector.data file> <path to yolo-custom.cfg file> <path to .weights file in backup folder> -dont_show`

### (9). Test mean average precision of specific weights

- `./darknet detector map <path to custom detector.data file> <path to yolo-custom.cfg file> <path to .weights file in backup folder> -dont_show`

例如:

`./darknet detector map ~/training/pill/detector.data ~/training/pill/cfg/yolov3custom.cfg  
~/training/pill/backup/yolov3custom_last.weights -dont_show`

### (10). Run your model

- Running the model on an image:

- `./darknet detector test <path to custom detector.data file> <path to yolo-custom.cfg file> <path to final.weights file in backup folder> -ext_output <path to appropriate image for testing>`

例如: `./darknet detector test ~/training/pill/detector.data`

`~/training/pill/cfg/yolov3custom.cfg ~/training/pill/backup/yolov3custom_last.weights -  
ext_output ~/training/pill/test/xxx.jpg`

- Running the model on a video:

- `./darknet detector demo <path to custom detector.data file> <path to yolo-`



custom.cfg file> <path to final.weights file in backup folder> -ext\_output <path to appropriate video for testing>

○ Running the model on a webcam:

○ ./darknet detector demo <path to custom detector.data file> <path to yolo-custom.cfg file> <path to final.weights file in backup folder> -c 0

\* If you want to save the predicted result images:

./darknet detector test <data file> <cfg file> <weights file> -ext\_output <image path> -thresh 0.1 -out\_filename /path/to/save/prediction.jpg